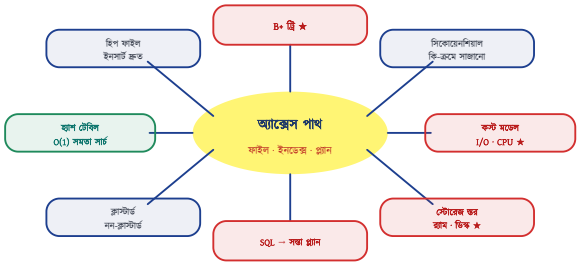
ফাইল অর্গানাইজেশন, ইনডেক্সিং ও কোয়েরি অপটিমাইজেশন

হিপ · সিকোয়েনশিয়াল · হ্যাশড · ইনডেক্সড · বি প্লাস ট্রি · হ্যাশ · কস্ট-বেজড অপটিমাইজার

এগারো পাতা: ফাইলের ভৌত সাজানো, বাইটগুলো কোথায় থাকে, ইনডেক্স কীভাবে সারি খুঁজে পায়, দুটি অ্যাক্সেস কাঠামো, অপটিমাইজার কীভাবে সস্তা প্ল্যান বাছে, এবং একটি সম্পূর্ণ হিসাব-কর্মের উদাহরণ।



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = অপর অ্যাক্সেস কাঠামো

সিলেবাসের যে অংশ

সিলেবাস এই লেসনের প্রতিটি অংশ নাম করে: "File organization and data storage; Indexing: primary and secondary indexes, B+ trees, hash tables; BCS Query optimization;" **তিনটি অংশই এখানে পূর্ণ করা হয়েছে।** db1-এ সম্পর্কগুলো বানানো হয়েছিল, db2-এ তার বীজগণিতা ও SQL, db3-এ রিডান্ড্যান্সি সরানো হয়েছিল, আর db5-এ ট্রানজেকশন ও কনকারেন্সি। এই সবের মাঝখানে এই লেসনের একটিমাত্র প্রশ্ন: **টেবিলগুলো সুন্দর ও পরিষ্কার — তবে তাদের বাইটগুলো কোথায়, আর কোটি সারির মধ্যে একটি সারিতে ডিবিএমএস কীভাবে পৌঁছায়?**

হাই-ইল্ড তালিকা

① হিপ: ক্রমহীন, ইনসার্ট দ্রুত
② সিকোয়েনশিয়াল: সাজানো, সার্চ দ্রুত
③ হ্যাশড: `h(k)=k mod N`
④ ইনডেক্সড: বাড়তি জায়গা, খুব দ্রুত
⑤ প্রাইমারি স্টোর = র‍্যাম, ভোলাপূর্ণ
⑥ বিট → বাইট → ব্লক → ফাইল → ডিবি
⑦ প্রাইমারি ইনডেক্স = সার্চ কি-তে
⑧ **মাত্র একটি ক্লাস্টার্ড ইনডেক্স ★**
⑨ ডেন্স = প্রতি রেকর্ডে, স্পার্স = প্রতি ব্লকে
⑩ B+ ট্রি: সব কি লিভে ★
⑧ হ্যাশ: =-এ O(1), রেঞ্জ নয়
⑫ প্ল্যান: SQL→পার্সার→অপটিমাইজার

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

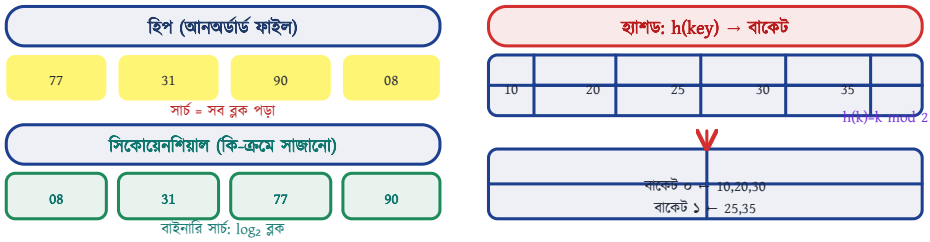
সূচিপত্র

✓ ফাইল অর্গানাইজেশন, চার প্রকার ... পৃ. ২
✓ স্টোরেজের ধরন ও ইউনিটের সিঁড়ি ... পৃ. ৩
✓ ইনডেক্স: চার প্রকার ... পৃ. ৪
✓ ইনডেক্সের ভেতরে: ডেন্স, স্পার্স ... পৃ. ৫
✓ B+ ট্রি ও লিভ চেইন ... পৃ. ৬
✓ হ্যাশ টেবিল ও কলিশন ... পৃ. ৭
✓ কোয়েরি অপটিমাইজেশন, ধাপে ধাপে ... পৃ. ৮
✓ কস্ট মডেল ও ইনডেক্স ডিজাইন ... পৃ. ৯
✓ রাতের পরীক্ষার পাতা ... পৃ. ১০
✓ অনুশীলনী MCQ + লিখিত ... পৃ. ১১

১ ফাইল অর্গানাইজেশন: রেকর্ড আসলে ডিস্কে কীভাবে থাকে

সংজ্ঞা: **ফাইল অর্গানাইজেশন** (File Organization) হলো ডেটা ফাইলের ভেতরে **রেকর্ডগুলো শারীরিকভাবে কীভাবে সংরক্ষিত হয়** — ডিস্কে তাদের ক্রম এবং একটি রেকর্ডে পৌঁছাতে ডিবিএমএস যে পথ অনুসরণ করে। এটি একটি *ভৌত* সিদ্ধান্ত: একই টেবিল চার রকমে রাখা যায়, আর চার রকমের insert, delete ও search খরচ সম্পূর্ণ ভিন্ন।

অর্গানাইজেশন	বর্ণনা ও বৈশিষ্ট্য
হিপ (আনঅর্ডার্ড)	রেকর্ড **একেবারে কোনো ক্রমে নয়**। ইনসার্ট দ্রুত — শুধু পিছনে যোগ করা। সার্চ ধীন, O(n): প্রতিটি ব্লক পড়তে হয়।
সিকোয়েনশিয়াল	রেকর্ড **একটি কি ফিল্ড অনুযায়ী সাজানো**। ওই কিতে সার্চ দ্রুত, কিন্তু ইনসার্ট ধীন — পরের সব রেকর্ড সরাতে হয়।
হ্যাশড	হ্যাশ ফাংশন স্টোরেজ অবস্থান বের করে, ফলে **ডাইরেক্ট অ্যাক্সেস**। সমতা সার্চ দ্রুত; **কলিশন সম্ভব**।
ক্লাস্টার্ড / ইনডেক্সড	ডেটা **ইনডেক্স পয়েন্টারের সঙ্গে** রাখা। সার্চ খুব দ্রুত; ইনডেক্সের জন্য **বাড়তি জায়গা** লাগে।



একই পাঁচটি রেকর্ড তিন রকমে রাখা — ডিস্কের ক্রমই আসল পার্থক্য

অর্গানাইজেশন	Insert	Delete	Search	Space
হিপ	**Fast**	Moderate	Slow	Minimal
সিকোয়েনশিয়াল	Slow	Slow	**Fast**	Minimal
হ্যাশড	Moderate	Moderate	**Fast**	Moderate
ইনডেক্সড	Moderate	Moderate	**Very Fast**	Extra

মনে রাখার কৌশল: **"ডিস্কের ক্রম সার্চ কেনে আর insert বিক্রি করে।"** হিপ কোনো ক্রম রাখে না, তাই কিছুই সরাতে হয় না — তাই তার insert Fast। সিকোয়েনশিয়াল ফাইল ক্রম ধরে রাখে, তাই নতুন কি এলে অর্ধেক ফাইল সরে যেতে পারে — তাই তার insert Slow। ফাঁকা ভাত নেই: ওই চার-কলামের টেবিলটিই এই বাণিজ্য, আর এটি টেবিল হিসেবেই **বারবার আসে**।

পরীক্ষার ফাঁদ ✍ তিনটি: **১.** "সিকোয়েনশিয়াল ইনসার্ট ধীন" মানে ধীন **কারণ রেকর্ড সরাতে হয়**। **২.** হ্যাশড সার্চ শুধু **সমতার জন্য** Fast — রেঞ্জের জন্য ক্রম ধরে কোনো সুবিধা নেই। **৩.** "ইনডেক্সড ফাইলে বাড়তি জায়গা লাগে না" এটি **মিথ্যা**; ইনডেক্স একটি আলাদা কাঠামো, তাই শুধু Indexed সারিতেই Extra লেখা।

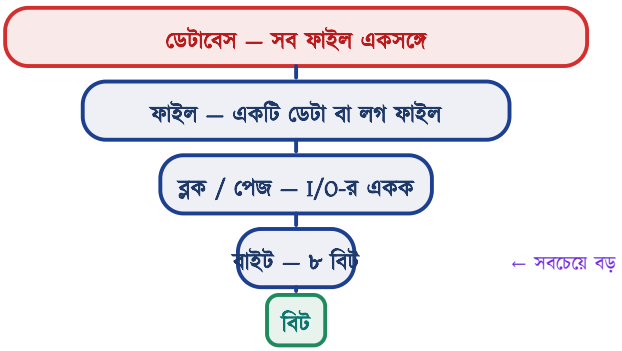
অধ্যায়ের অন্যান্য কাজ: db1-এ সম্পর্ক বানানো, db2-এ বীজগণিতা ও SQL, db3-এ রিডান্ড্যান্সি সরানো। এই লেসন অন্য প্রশ্ন করে: **বাইটগুলো কোথায়, আর একটি সারিতে কীভাবে পৌঁছাব ডিবিএমএস?**

২ ডেটা স্টোরেজের ধরন ও স্টোরেজ ইউনিটের সিঁড়ি

বাইটগুলো শারীরিকভাবে কোথায় আছে, আর ফাইলের ভেতরে কীভাবে সাজানো — এই দুটি আলাদা প্রশ্ন। ডিবিএমএস কাজের ডেটা **তিন স্তরে** রাখে, আর প্রতিটি স্তরের ভেতরে ডেটাকে **পাঁচটি ইউনিটে** চিহ্নিত করে।

স্টোরেজের ধরন	বর্ণনা	মনে রাখুন
প্রাইমারি (র‍্যাম)	সবচেয়ে দ্রুত — সিপিইউ সরাসরি পড়ে। **ভোলাপূর্ণ:** বিদ্যুৎ চলে গেলে ডেটা হারিয়ে যায়।	বাফার পুল, টেম্প টেবিল
সেকেন্ডারি (HDD / SSD)	**স্থায়ী (Persistent)** — বিদ্যুৎ চলে গেলেও থাকে। র‍্যামের চেয়ে ধীন; ডেটা ফাইল, লগ ফাইল ও ইনডেক্স এখানেই।	ডেটাবেসের নিজের ঠিকানা
টার্শিয়ারি (টেপ / CD)	খুব ধীন, প্রায় কেবল ধারাবাহিক অ্যাক্সেস। **আর্কাইভিং** ও ব্যাকআপে ব্যবহৃত।	অফলাইন ঠান্ডা কপি

সেকেন্ডারি ডিস্ক	HDD	SSD
মূল অংশ	চলমান ডিস্ক ও রিডিং হেড	ফ্ল্যাশ মেমোরি কোষ (Flash cell)
অ্যাক্সেস	**সিক + রোটেশন লেটেন্সি** — ডিস্ক ড়ালার রোটেখন্ড সময় লাগে	সিক নেই, সরাসরি র‍্যান্ডম অ্যাক্সেস
দ্রুততা	ধীন, তবু স্পষ্ট ধীন	**অনেক বেশি** — ডিস্ক I/O নির্ভরতা নূন্য
দুর্বলতা	মেকানিক্যাল ক্লান্তি, ঘরোয়া বাসস্থাপন	সীমিত রাইট লাইফ — বারবার লেখা ধীর হয়



যে সিঁড়িতে প্রতিটি I/O খরচ গোনা হয়

সিঁড়িটি কথায়: **বিট → বাইট → ব্লক/পেজ → ফাইল → ডেটাবেস**। বাইট = ৮ বিট; **ব্লক (পেজ)** হলো ডিস্ক থেকে ডিবিএমএস আসলে যে নির্দিষ্ট আকারের এককে পড়ে — সাধারণত 4KB বা 8KB; ফাইল হলো ডিস্কে ব্লকের নামযুক্ত ধারা; আর ডেটাবেস হলো সব ফাইল একসঙ্গে। **প্রতিটি ডিস্ক I/O গোনা হয় ব্লকে, বাইটে নয়** — তাই ব্লকিং ফ্যাক্টর এত গুরুত্বপূর্ণ।

ব্লকিং ফ্যাক্টর ✍️ যে সংখ্যাটা হিসাব করতে হবে: একটি ব্লকে কতগুলো রেকর্ড ঢোকে, তাকেই বলে blocking factor: `bfr = ⌊ ব্লকের আকার ÷ রেকর্ডের আকার ⌋` ফাইল জুড়ে ব্লক সংখ্যা `⌈ রেকর্ড ÷ bfr ⌉`। Unspanned মানে কোনো রেকর্ড দুটি ব্লকের মাঝে ভাঙা নয়। ব্লকিং ফ্যাক্টরে **ফ্লোর**, ব্লক সংখ্যায় **সিলিং** নিতে হবে।

পরীক্ষার ফাঁদ ✍️ চারটি: ১. "**প্রাইমারি** স্টোরেজ" মানে র‍্যাম — না প্রাইমারি কি, না প্রথম স্তর — আর র‍্যাম ভোলাপূর্ণ। ২. SSD হলো **সেকেন্ডারি** স্টোরেজ: যত দ্রুতই হোক, স্থায়ী। ৩. টার্শিয়ারি মানে **আর্কাইভিং**, আর তা প্রায় ধারাবাহিক — টেপের ভেতরে গিয়ে খুঁজে বের করা যায় না। ৪. নামতে নামতে গতি ও স্থায়িত্ব দুটোই কমে: প্রাইমারি দ্রুততম ও বিস্মৃতিশীল, টার্শিয়ারি সবচেয়ে ধীন ও স্থায়ী।

মনে রাখার কৌশল: "**প্রাইমারি দ্রুত কিন্তু বিস্মৃতিশীল; সেকেন্ডারি ডেটাবেসের নিজের ঠিকানা; টার্শিয়ারি আটলার।**" আর সিঁড়ির জন্য: "**অনেক বিট মিলে বাইট, অনেক বাইট মিলে ব্লক, অনেক ব্লক মিলে ফাইল, অনেক ফাইল মিলে ডেটাবেস।**" প্রতিটি ধাপ হলো "নিচেরটার অনেকগুলো", আর ব্লকই একমাত্র একক যা হার্ডওয়্যার একবারে সরায় — তাই কস্ট মডেল অক্ষর গোনে, পাতা গোনে নয়।

৩ ইনডেক্স: সংজ্ঞা ও চার প্রকার

সংজ্ঞা: **ইনডেক্স (Index)** হলো এমন একটি **ডেটা স্ট্রাকচার যা পুনরুদ্ধার দ্রুত করে** — এটি সার্চ কি থেকে সারির অবস্থান পর্যন্ত একটি অতিরিক্ত সাজানো মানচিত্র রাখে। এটি ডেটা ধরে না — এটি ডেটার **পথ** ধরে, ফলে একটি সার্চ সব ব্লক নয়, খুব অল্প কিছু ব্লক স্পর্শ করে।

ইনডেক্সের ধরন	বর্ণনা
প্রাইমারি ইনডেক্স	**প্রাইমারি (সার্চ) কি**-র উপর বানানো। কি টি অদ্বিতীয় ও অপরিবর্তিত হওয়ায় ইনডেক্সটি **ডেটা ফাইলকে শারীরিকভাবে ক্রমে সাজায়** — রেকর্ডগুলো ডিস্কেই কি-ক্রমে থাকে।
সেকেন্ডারি ইনডেক্স	**নন-কি কলামের** উপর বানানো (ফরেন কি, বা যে কলামে প্রায়ই ফিল্টার করেন)। এটি একটি **বিকল্প পথ** দেয়; ডেটা নিজে সাজানো হয় না, তাই কি-র মান পুনরাবৃত্ত হতে পারে।
ক্লাস্টার্ড ইনডেক্স	**ডেটা ইনডেক্স কি-র উপর শারীরিকভাবে সাজানো**। কি-ক্রমে কোনো রেঞ্জ পড়া তখন একটি সরল ধারাবাহিক স্ক্যান।
নন-ক্লাস্টার্ড ইনডেক্স	ডেটার দিকে একটি **লজিক্যাল পয়েন্টার**, যা আলাদা কাঠামোয় থাকে; সারি যেখানে ছিল সেখানেই থাকে, ইনডেক্স শুধু তার দিকে তাকায়।

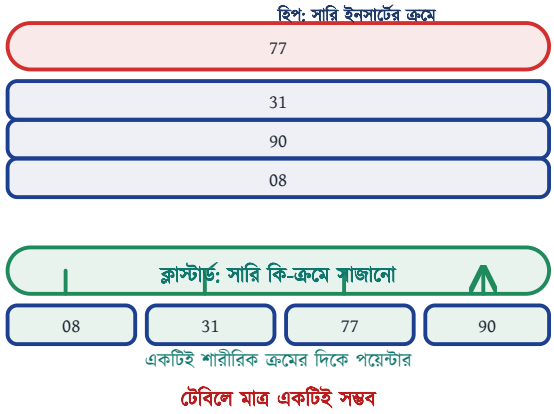
কোথায় ইনডেক্স দেব	কেন তা বলা হয়
WHERE, JOIN, ORDER BY-এর কলাম	সাধারণত ফিল্টার ও সাজানো হয় যেখানেই
সবচেয়ে **সিলেক্টিভ** কলাম **আগে**	বেশি ভিন্ন মান = ছোট ইনডেক্স, তুলনীয় রেচ, বাস্তব সিলেক্ট
যা কারুক্ষ্য পরীক্ষায় নয় কলাম	স্থান ঝবে, বা ব্যবহার না হলে বেসস

পরীক্ষার ফাঁদ ✍ ইনডেক্স বিনামূল্যে নয়: ইনডেক্স বাড়ার সাথে **মূল্য বাড়ে**। প্রতিটি INSERT/UPDATE/DELETE-এ ইন্ডেক্সটিও বদলাতে হয়, **ইন্ডেক্স রক্ষাকে ব্যবহার করে**, কাজের লেখা-প্রধান টেবিলে বেশি ইন্ডেক্স ডেটাবেজকে ধীন করে।

পরীক্ষার ফাঁদ ✍ যেটা অবশ্যই মুখস্থ: একটি টেবিলে সর্বোচ্চ একটিই ক্লাস্টার্ড ইনডেক্স (Clustered Index) **থাকতে পারে।** ক্লাস্টার্ড ইনডেক্স ঠিক করে দেয় *সারিগুলো কোন শারীরিক ক্রমে জমা থাকবে*, আর একই সারি-সমূহ একসঙ্গে একাধিক ক্রমে জমা রাখা যায় না — তাই দ্বিতীয় ক্লাস্টার্ড ইনডেক্সের ঠিক করার কিছুই অবশিষ্ট থাকে না। তবে একটি টেবিলে **যত খুশি নন-ক্লাস্টার্ড (সেকেন্ডারি) ইনডেক্স** থাকতে পারে, প্রতিটি ফিল্টার করা কলামের জন্য একটি। "দুটি", "প্রতিটি কলামে একটি" ও "যেকোনো সংখ্যক" — ক্লাস্টার্ডের ক্ষেত্রে সবগুলোই ভুল।

প্রাইমারি বনাম ক্লাস্টার্ড — গুলিয়ে ফেলবেন না: দুটি শব্দ একসঙ্গে চলে, কিন্তু পাঠ্যপ্রশ্নে সমার্থক নয়। **প্রাইমারি ইনডেক্স** কোন *কি-র* কথা বলে (সার্চ কি, আর ফাইল সেই ক্রমে সাজানো)। **ক্লাস্টার্ড ইনডেক্স** কোনটি *শারীরিকভাবে সত্য* তা বলে (সারি সেই ক্রমেই বসে)। **সেকেন্ডারি** ইনডেক্স কখনো ফাইল সাজায় না, তাই শুধু সেকেন্ডারি ইনডেক্স থাকা টেবিল ডিস্কে আসলে একটি হিপ।

নন-ক্লাস্টার্ড লিভ এন্ট্রিতে থাকে কী: <কি-র মান, রো লোকেটর>। লোকেটর হলো সারির **প্রাইমারি কি**, যদি থাকে; আর প্রাইমারি কি যদি সার্ভোয়েট (স্বয়ংক্রিয়ভাবে বানানো আইডি) হয়, তবে লোকেটর হলো RID — রেকর্ড আইডেন্টিফায়ার, মোটামুটি তার শারীরিক ঠিকানা। লোকেটর অনুসরণ করার পরেই টেবিল স্পর্শ হয়।

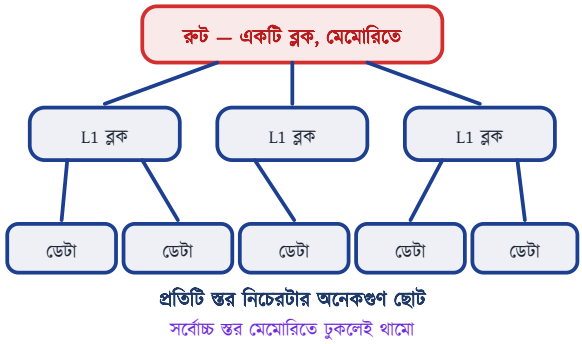


ক্লাস্টার্ড ইনডেক্স সারি নাড়ে; নন-ক্লাস্টার্ড শুধু পয়েন্টার

৪ ইনডেক্সের ভেতরে: ডেন্স, স্পার্স, মাল্টিলেভেল — ও এর দাম

ধরন	এন্ট্রি একটির জন্য...	আকার	সার্চ খরচ
ডেন্স ইনডেক্স	প্রতি **রেকর্ডের** জন্য একটি	বড় — ফাইলের সঙ্গে বাড়ে	ছোট ইনডেক্স, দ্রুততম লুকআপ
স্পার্স ইনডেক্স	প্রতি **ডেটা ব্লকের** জন্য একটি — ঐ ব্লকের প্রথম কি-র দিকে	ছোট — শুধু ব্লকের সঙ্গে বাড়ে	বড় ইনডেক্স, তবে সার্চের ব্লক অনেক কম

মাল্টিলেভেল ইনডেক্স — ইনডেক্সের উপর ইনডেক্স: স্পার্স ইনডেক্সকেও খুঁজতে হয়, আর সেটি মেমোরিতে না থাকলে তার *নিজের* ব্লক পড়তে হয়। তাই ডিবিএমএস ইনডেক্সের **উপরে দ্বিতীয় স্তর** বানায়, তারপর তৃতীয়, যতক্ষণ না সর্বোচ্চ স্তর একটিমাত্র ব্লক হয়ে যায় যা সবসময় মেমোরিতে থাকে। একটি **মাল্টিলেভেল (B-tree / B+ tree) ইনডেক্স** আসলে এটিই: সর্বোচ্চ স্তর মেমোরিতে, আর প্রতিটি নিচের স্তর ফ্যান-আউট অনুযায়ী ছোট হতে হতে যায়। তাই ১০ কোটি রেকর্ডের সার্চেও মাত্র কয়েকটি ডিস্ক পড়া লাগে।



মাল্টিলেভেল: ইনডেক্স নিজেই একটি খোঁজার উপযোগী ট্রি

পরীক্ষার ফাঁদ ✍️ স্পার্সের জন্য সাজানো ফাইল লাগে: **স্পার্স** ইনডেক্স প্রতি ব্লকে একটি এন্ট্রি রাখে, তাই তার এন্ট্রি বলতে হবে "এই ব্লকের *প্রথম* কি এটা" — কথাটি কেবল তখনই অর্থপূর্ণ যখন ব্লকগুলো **কি-ক্রমে** সাজানো। তাই স্পার্স ইনডেক্স **সাজানো (সিকোয়েনশিয়াল) ফাইলের** উপর বানানো হয়। **ডেন্স** ইনডেক্সে প্রতি রেকর্ডে একটি এন্ট্রি থাকে, তাই সেটি **আনঅর্ডার্ড হিপ**-এর উপরও বানানো যায়। বাণিজ্য একেবারে সরল: ডেন্স বেশি জায়গা নেয় ও কম পড়া লাগায়; স্পার্স কম জায়গা নেয় ও বেশি পড়া লাগায়।

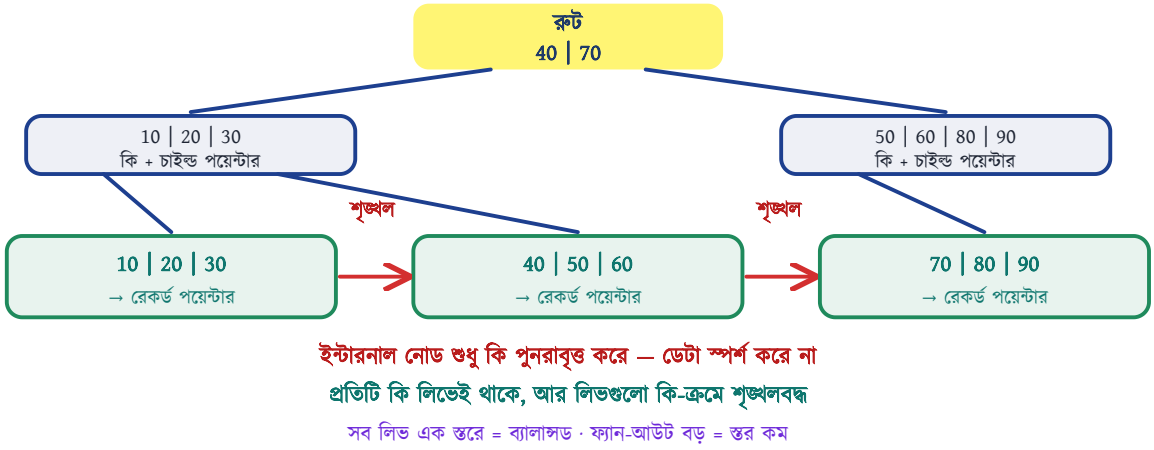
প্রতিটি ইনডেক্সের যে দাম: ১. বাড়তি জায়গা — ইনডেক্স হলো কি-এর দ্বিতীয় অনুলিপি। ২. প্রতিটি INSERT ও DELETE-কেও ইনডেক্স আপডেট করতে হয়, তাই লেখা ধীর হয়। ৩. প্ল্যান বাছার সময় অপটিমাইজার প্রতিটিকে বিবেচনা করে। ৪. অব্যবহৃত ইনডেক্স নিছক খরচ। **অতি-ইনডেক্সিং** — প্রতিটি কলামে ইনডেক্স — একটি সত্যিকারের ফাঁদ: লেখা ধীর হয়, জায়গা নষ্ট হয়, অপটিমাইজারও বিভ্রান্ত হয়, কোনো লাভ ছাড়াই।

মনে রাখার কৌশল: **"ইনডেক্স হলো দামের ট্যাগ, ডেটার অনুলিপি নয়।"** এতে কি আছে আর লোকেটর আছে, ডেটা নয়। আর **"ডেন্স লম্বা ও চওড়া, স্পার্স সরু ও লম্বা"** — ডেন্সে কম ব্লকে অনেক এন্ট্রি; স্পার্সে কম এন্ট্রি, কিন্তু বেশি লম্বা ট্রি, তবে মাল্টিলেভেল স্পার্স ইনডেক্সে ফারাকটা অনেকটাই কমে যায়। শুধু কলাম থাকার কারণে ইনডেক্স কখনো বানাবেন না: **যেটাতে ফিল্টার, জয়েন বা সর্ট করেন।**

ইনডেক্স শারীরিকভাবে থাকে কোথায়: ইনডেক্স ডেটা ফাইলের **ভেতরে** নয়: এটি একই ডিস্কে আলাদা ব্লকের একটি সেট, আর ডিবিএমএস সবচেয়ে গরম স্তরগুলো বাফার পুলে RAM-এ রাখতে পারে। মাল্টিলেভেল ইনডেক্স দ্রুত হওয়ার পুরো কারণ এটাই — ট্রির সর্বোচ্চাংশ কখনো ডিস্কে যায় না। তাই সার্চের খরচ কয়েকটি RAM পড়া আর সর্বোচ্চ এক-দুটি ডিস্ক পড়া, টেবিল যত বড় হোক।

৫ বি প্লাস ট্রি (B+ Tree): কাজের ইনডেক্স, আর রেঞ্জ দ্রুত করা লিভ চেইন

সংজ্ঞা: **বি প্লাস ট্রি (B+ Tree)** হলো ডেটাবেসের সবচেয়ে সাধারণ ইনডেক্স কাঠামো: একটি **ব্যালান্সড ট্রি**, যেখানে **প্রতিটি লিভ একই স্তরে** বসে। ইন্টারনাল নোডে থাকে শুধু **কি + চাইল্ড পয়েন্টার**, আর সেটি কেবল সার্চের পথ দেখায়; লিভ নোডে থাকে **কি + রেকর্ড পয়েন্টার**। ট্রিটি ব্যালান্সড এবং তার **ফ্যান-আউট বেশি** — একটি নোডে শত শত কি — তাই ট্রি মাত্র কয়েক স্তর গভীর, ফলে search, insert ও delete তিনটিই O(log n) এবং খরচ ক্ষুদ্র।



সঠিক বি প্লাস ট্রি: সব লিভ এক স্তরে, সব কি লিভে, লিভ বাম থেকে ডানে শৃঙ্খলবদ্ধ

রেঞ্জ কোয়েরিতে কেন জেতে ✍️ প্রমিত উত্তর: বি প্লাস ট্রিতে **প্রতিটি কি তার রেকর্ড পয়েন্টারসহ লিভেই সংরক্ষিত**, আর **লিভগুলো কি-ক্রমে শৃঙ্খলে সাজানো**। তাই `WHERE salary BETWEEN 20000 AND 30000`-এর ক্ষেত্রে ইনডেক্স প্রথমে 20000-এর **প্রথম লিভ খুঁজে বের করে** — একটি সাধারণ ট্রি-ডিসসেন্ট — $O(\log n)$ খরচে, তারপর সহজেই **লিভ চেইন ধরে পরের লিভ ধরে ধরে** এগিয়ে যায় যতক্ষণ না 30000 পেরিয়ে যায়। ইন্টারনাল নোড ওই প্রথম ডিসসেন্ট ছাড়া আর কোনো কাজ করে না।
তাই বি প্লাস ইনডেক্সে রেঞ্জ কোয়েরির খরচ $O(\log n) + k$, যেখানে k হলো মিলে যাওয়া এন্ট্রির সংখ্যা।

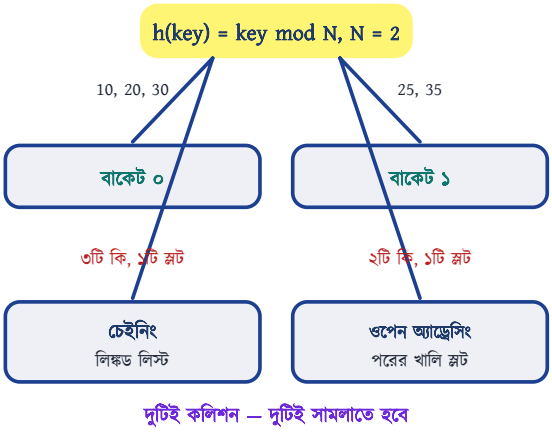
পরীক্ষার ফাঁদ ✍️ বি-ট্রি বনাম বি প্লাস ট্রি — মেরে ফেলার মতো তিনটি ভুল উত্তর: "এর **কোনো ইন্টারনাল নোড নেই**" — মিথ্যা, ইন্টারনাল নোড আছে ও ডিসসেন্ট দিকনির্দেশ করে। "**সব রেকর্ড রুটে রাখে**" — মিথ্যা, রুটে শুধু কি ও চাইল্ড পয়েন্টার। "**এটি সবসময় বাইনারি ট্রি**" — মিথ্যা, ফ্যান-আউট বেশি বলে একটি নোডে শত শত কি ও হাজার হাজার চাইল্ড থাকে। একমাত্র সঠিক বাক্য হলো **লিভে প্রতিটি কি থাকে এবং লিভগুলো কি-ক্রমে শৃঙ্খলবদ্ধ**। সাধারণ **বি-ট্রিতে** রেকর্ড *সব* নোডে বসে, তাই রেঞ্জ স্ক্যানে স্তর বদলাতে হয়; **বি প্লাস ট্রিতে** শুধু লিভে রেকর্ড থাকে, তাই রেঞ্জ স্ক্যান একটানা একটি হাঁটা।

বিষয়	বি-ট্রি	বি প্লাস ট্রি
ডেটা কোথায় বসে	**প্রতিটি** নোডে, ইন্টারনালসহ	**শুধু লিভে**
রেঞ্জ কোয়েরি	স্তর বদলাতে হয়	**লিভ চেইন ধরে এক হাঁটা**
বাস্তবে কোথায়	ফাইল সিস্টেমে বেশি	**ডেটাবেস ইনডেক্সে প্রায় সবসময়**

৬ হ্যাশ টেবিল (Hash Table): সমতায় O(1) — আর রেঞ্জ কেন তাকে মেরে ফেলে

সংজ্ঞা: **হ্যাশ টেবিল** স্টোরেজ অবস্থান *সরাসরি* বের করে: একটি **হ্যাশ ফাংশন** কি-কে একটি বাকেটে ম্যাপ করে, তাই কোনো তুলনা বা ট্রাভার্সাল লাগে না। `h(key) = key mod N`, যেখানে **N হলো বাকেটের সংখ্যা**।

সমাধান-কৃত উদাহরণ ✍️ N = 2 বলা আছে: এখানে N = 2, তাই মোট দুটি বাকেট এবং h(key) = key mod 2:
10 → 0 · 20 → 0 · 25 → 1 · 30 → 0 · 35 → 1
তাই **বাকেট ০ = [10, 20, 30]** এবং **বাকেট ১ = [25, 35]**। ১০, ২০ ও ৩০ তিনটিই ০-তে হ্যাশ হয় — এটিই একটি **কলিশন**: ভিন্ন কি একই স্লটের জন্য স্পর্ধা করছে। এই সব মান **কেবল** N = 2 হলে ঠিক; N = 10 হলে উত্তর একেবারে বদলে যেত, তাই **গণনার আগে অবশ্যই N বলে দাও।**



মাত্র দুটি বাকেট — কি হয়তো কলিশন করে, নয়তো করে না

কলিশন সামলানোর পদ্ধতি	কীভাবে কাজ করে
চেইনিং (Chaining)	প্রতিটি বাকেটে একটি **লিঙ্কড লিস্ট**; কলিশন করা রেকর্ডটি ঐ বাকেটের তালিকায় যোগ হয়, আর সার্চে ছোট তালিকাটি হেঁটে যায়। উপরের বাকেট ০ হলো চেইন 10 → 20 → 30।
ওপেন অ্যাড্রেসিং (Open Addressing)	কোনো তালিকা নেই। গণনা করা স্লট ভর্তি থাকলে **পরের খালি স্লটের দিকে প্রোব** করে রেকর্ডটি রাখা হয়। টেবিল ভরে গেলে আরও বড় করে রিহ্যাশ করতে হয়।

হ্যাশ বনাম বি প্লাস ট্রি: হ্যাশ: সমতা লুকআপ **গড়ে O(1)** — এর চেয়ে ভালো আর কিছু নেই। কিন্তু হ্যাশ ফাংশন ক্রম ধ্বংস করে, তাই **রেঞ্জ কোয়েরি, ORDER BY ও MIN/MAX নিরপেক্ষ। বি প্লাস ট্রি:** একক সমতা হিটে একটু ধীর, কিন্তু কি-গুলো ক্রমে থাকে, তাই রেঞ্জ, সর্টিং ও "পরের রেকর্ড" সবই স্বাভাবিক। **বড় ও স্থিতিশীল কি-তে exact match**-এর জন্য হ্যাশ বাছো; কোয়েরিতে রেঞ্জ বা ক্রম লাগলেই বি প্লাস ট্রি।

পরীক্ষার ফাঁদ ✍️ তিনটি: ১. হ্যাশ ইনডেক্স "salary = 25000" এক ধাপে উত্তর দেয়, কিন্তু "salary BETWEEN 20000 AND 30000" সব পড়া ছাড়া উত্তর দিতে **পারে না** — তাই বেশি আসা জোড়া হলো **BETWEEN-এ বি প্লাস ট্রি, =-এ হ্যাশ। ২.** "হ্যাশ টেবিলে কলিশন হয় না" — দুটি কি একই জায়গায় গেলেই মিথ্যা। **৩.** কলিশন কমাতে N বেশি হওয়া উচিত, কিন্তু বড় N জায়গা নষ্ট করে — বাস্তবায়ন N হিসাব করে নেয় এবং ফাইল বড় হলে রিহ্যাশ করে।

৭ কোয়েরি অপটিমাইজেশন: SQL লেখা থেকে এক্সিকিউশন প্ল্যান পর্যন্ত

সংজ্ঞা: কোয়েরি অপটিমাইজেশন (Query Optimization) হলো SQL কোয়েরির জন্য **সবচেয়ে কার্যকর এক্সিকিউশন প্ল্যান বাছার** প্রক্রিয়া। SQL বলে *কী* চাও, কিন্তু একই সঠিক সারি দেখানোর প্ল্যান অনেকগুলো থাকতে পারে, আর অপটিমাইজার সবচেয়ে কম খরচেরটি বেছে নেয়।



প্রতিটি ডিবিএমএস এই পাঁচটি ধাপই অনুসরণ করে

কৌশল	বর্ণনা
ইনডেক্স ব্যবহার	WHERE, JOIN ও ORDER BY-তে ব্যবহৃত প্রতিটি কলামে ইনডেক্স রাখো — ইনডেক্স কেবল এই তিনটি জায়গাতেই কোনোদিন ব্যবহৃত হতে পারে।
SELECT * এড়িয়ে চলো	শুধু দরকারি কলাম বেছে নাও। অপ্রয়োজনীয় কলামে বাড়তি I/O লাগে *আর* কোয়েরি কেবল কভারিং ইনডেক্স দিয়েই সমাধান হওয়া বন্ধ হয়ে যায়।
কার্যকর জয়েন	বড় অসাজানো ইনপুটে **হ্যাশ জয়েন** · দুই পাশই সাজানো থাকলে **মার্জ জয়েন** · এক পাশ খুব ছোট হলে **নেস্টেড লুপ**।
সাবকোয়েরি এড়াও	IN (SELECT …) কে JOIN বা EXISTS-এ লেখো; তাহলে অপটিমাইজার জয়েনের কৌশল বাছতে পারে, আর প্রতিবার ভেতরের কোয়েরি চালাতে হয় না।
কোয়েরি হিন্ট	DBMS-নির্দিষ্ট নির্দেশনা, কোয়েরির ভেতরেই লেখা, যাতে পরিসংখ্যানভিত্তিক অনুমান ভুল হলে অপটিমাইজারকে দিকনির্দেশ দেওয়া যায়।
পার্টিশনিং	একটি বড় টেবিলকে ভাগে ভাগ করা, যাতে কোয়েরি শুধু দরকারি ভাগই স্পর্শ করে।
ক্যাশিং	**ম্যাটেরিয়ালাইজড ভিউ** কোয়েরির ফলাফল সংরক্ষণ করে যাতে সরাসরি পড়া যায়; কোয়েরি ক্যাশ প্লান ও তার ফলাফল পুনর্ব্যবহার করে।

পরীক্ষার ফাঁদ ✍ দুটি: ১. অপটিমাইজার কোয়েরি **কী দেয়** তা বদলায় না — সে বদলায় উত্তর **কীভাবে** তৈরি হয়। একই সারি দেখানো দুটি প্ল্যানই সঠিক; একটি সস্তা। ২. "অপটিমাইজার ধীর কোয়েরি বেছে নিয়েছে" সাধারণত মানে **পরিসংখ্যান পুরনো** বা **একটি দরকারি ইনডেক্স নেই** — অপটিমাইজার নিজে ভাঙা নয়।

মনে রাখার কৌশল: **"যেটাতে ফিল্টার, জয়েন বা সর্ট করো — এই তিনটি শব্দই পুরো তালিকা।"** অন্য কোনো কলামের ইনডেক্স ওই কোয়েরিতে কোনোদিন ব্যবহৃত হয় না, যত সুন্দরই হোক। আর "SELECT * **বাদ দিও**" শুধু পরিষ্কার-রাখার জন্য নয়: এটি অপ্রয়োজনীয় I/O এবং কভারিং-ইনডেক্স-একমাত্র প্ল্যান হারানো — দুটোই।

প্ল্যান পড়া ✍: বাস্তবের প্রতিটি ডিবিএমএস নিজের বেছে নেওয়া প্ল্যান দেখায় — বেশিরভাগ ক্ষেত্রে EXPLAIN — আর প্ল্যান কোনো গোপন জিনিস নয়। সেখানে ক্রমানুসারে থাকে কোন টেবিল স্ক্যান বা ইনডেক্স হবে, **জয়েনের ক্রম**, এবং প্রতিটি ধাপে ব্যবহৃত **জয়েন অ্যালগরিদম**। লিখিত প্রশ্নে "প্ল্যানটি প্রথমে Employee টেবিলের পূর্ণ টেবিল স্ক্যান করে, তারপর Department-কে তার প্রাইমারি কি-তে ইনডেক্স করে" — এটাই সম্পূর্ণ সঠিক উত্তর; নম্বর আসে শারীরিক কাঠামোর নাম বলা থেকে, SQL আবার লেখা থেকে নয়।

৮ কস্ট-বেজড বনাম হিউরিস্টিক অপটিমাইজেশন — ও প্ল্যান ভালো করার উপায়

কস্ট মডেল ✍ তিনটি বিষয়: প্রতিটি প্রার্থী প্ল্যানের জন্য অপটিমাইজার হিসাব করে:
১. I/O খরচ — **ডিস্ক পেজ কতবার পড়া ও লেখা হবে**। এটিই আধিপত্য করে, কারণ একটি ডিস্ক রিড সিপিইউ অপারেশনের কোটি গুণ ধীন।
২. CPU খরচ — কতগুলো তুলনা, টিউপল অপারেশন ও এক্সপ্রেশন হিসাব হবে।
৩. কমিউনিকেশন খরচ — সাইটগুলোর মধ্যে কত ডেটা যাবে; কেবল **ডিস্ট্রিবিউটেড** ডেটাবেসে গোনা হয়।

কস্ট-বেজড (স্ট্যাটিস্টিক্স) অপটিমাইজার: পরিসংখ্যান — প্রতি টেবিলে কত সারি, কত ভিন্ন মান, ডেটার বণ্টন — থেকে প্রতিটি প্রার্থী প্ল্যানের I/O ও CPU খরচ অনুমান করে এবং **সবচেয়ে সস্তাটি** বাছে। শক্তিশালী, কিন্তু সম্পূর্ণ নির্ভরশীল — পরিসংখ্যান হালনাগাদ থাকলেই কাজ করে।

হিউরিস্টিক (রুল-বেজড) অপটিমাইজার: ডেটা মেপে না দেখেই **নির্দিষ্ট রূপান্তর নিয়ম** প্রয়োগ করে: যত তাড়াতাড়ি সারি কমাতে SELECT **আগে**; যত তাড়াতাড়ি কলাম কমাতে projection **আগে**; **সবচেয়ে সীমাবদ্ধ জয়েনটি আগে**। এটি মাপতে পারে না, শুধু রূপান্তর করতে পারে — আধুনিক সিস্টেমে এই নিয়মগুলো কস্ট-বেজড অপটিমাইজেশনের উপরে বসানো থাকে।

ইনডেক্স সিলেক্টিভিটি ও কলামের ক্রম: **সিলেক্টিভিটি** হলো কোনো শর্ত কত ভাগ সারি টিকিয়ে রাখে। যে শর্ত **অত্যন্ত সিলেক্টিভ** — অল্প সারি বেঁচে থাকে — তার জন্যই ইনডেক্স লুকআপ লাভজনক। **কম্পোজিট ইনডেক্সে বেশি সিলেক্টিভ কলামটি প্রথমে** রাখো, কারণ সেটিই সার্চ সংকুচিত করে।

ব্লকিং ফ্যাক্টরের সম্পূর্ণ উদাহরণ ✍️ প্রতিটি ধাপ সহ: ডেটা ফাইলে **100 বাইটের 30,000টি রেকর্ড**, **1024 বাইটের ব্লকে** unspanned (কোনো রেকর্ড দুই ব্লকের মাঝে ভাঙা নয়), কি-র উপর সাজানো।
১. ডেটা ব্লকিং ফ্যাক্টর = ⌊1024 ÷ 100⌋ = 10 রেকর্ড/ব্লক ⇒ ডেটা ব্লক = 30000 ÷ 10 = 3000
২. স্পার্স প্রাইমারি ইনডেক্সে প্রতি ডেটা ব্লকে একটি এন্ট্রি ⇒ মোট **3000** এন্ট্রি; প্রতিটি এন্ট্রি **15 বাইট** (9 বাইট কি + 6 বাইট ব্লক পয়েন্টার)
৩. ইনডেক্স ব্লকিং ফ্যাক্টর = ⌊1024 ÷ 15⌋ = 68 এন্ট্রি/ব্লক
৪. ইনডেক্স ব্লক = ⌈3000 ÷ 68⌉ = ⌈44.1⌉ = 45 — **সিলিং** লাগে কারণ শেষ ব্লক আংশিক ভর্তি।
৫. বিপরীতে **ডেন্স** ইনডেক্সে 30,000 এন্ট্রি ⇒ ⌈30000 ÷ 68⌉ = ⌈441.2⌉ = 442 ব্লক — প্রায় দশগুণ বেশি।

পরীক্ষার ফাঁদ ✍️ বাম-ডান প্রিফিক্স নিয়ম: কম্পোজিট ইনডেক্স (a, b) WHERE a = ? এবং WHERE a = ? AND b = ? — দুটোতেই কাজ করে, কিন্তু শুধু WHERE b = ? একা কাজ করে না — ইনডেক্সটি নিজে থেকে b-এর উপর সাজানো নয়, তাই b-এর প্রতিটি মান পুরো ট্রিতে ছড়িয়ে আছে। এটিই **বাম-ডান প্রিফিক্স নিয়ম**, আর এটি **বারবার আসে**। একই নিয়ম বলায় ORDER BY a, b এই ইনডেক্সে ফ্রি, কিন্তু ORDER BY b ফ্রি নয়।

কভারিং ইনডেক্স — SELECT * বাদ দেওয়ার সবচেয়ে বড় কারণ:
যদি **কোয়েরির দরকারি প্রতিটি কলামই ইনডেক্সে থাকে**, অপটিমাইজার টেবিল **একেবারে না ছুঁয়েই** উত্তর দেয়; এটাই ইনডেক্স-অনলি স্ক্যান। না-চাওয়া একটি কলাম যোগ করা — SELECT *-এর অভ্যাস — কভারিং ইনডেক্স ধ্বংস করে। তাই কভারিং ইনডেক্স অল্প লেখার বিনিময়ে বড় পড়ার সাশ্রয় দেয়।

আগের পাতার সূত্রগুলো এক নজরে: **১. একটি টেবিলে সর্বোচ্চ একটিই ক্লাস্টার্ড ইনডেক্স** — একই সারি একাধিক শারীরিক ক্রমে জমা রাখা যায় না, তবে নন-ক্লাস্টার্ড ইনডেক্স যত খুশি। **২. ডেন্স** = প্রতি রেকর্ডে এন্ট্রি; **স্পার্স** = প্রতি ব্লকে এন্ট্রি, আর স্পার্সের জন্য অবশ্যই **সাজানো ফাইল** লাগে (এন্ট্রি বলে "ঐ ব্লকের প্রথম কি")। **৩. মাল্টিলেভেল** ইনডেক্স = ইনডেক্সের উপর ইনডেক্স, সর্বোচ্চ স্তর সবসময় RAM-এ। **৪.** নন-ক্লাস্টার্ড লিভ এন্ট্রি = <কি-র মান, রো লোকেটর> — লোকেটর প্রাইমারি কি, না হলে RID। **৫.** ইনডেক্সের দাম: বাড়তি জায়গা, এবং প্রতিটি INSERT ও DELETE-কেও তা আপডেট করতে হয়; অব্যবহৃত ইনডেক্স নিছক খরচ, আর কম্পোজিট ইনডেক্স **বাম-ডান প্রিফিক্স নিয়ম** মানে — a ও a, b দেয়, b একা নয়।

পরীক্ষার আগের রাতের শীট

তথ্যপাতা:

হিপ: ক্রমহীন, ইনসার্ট Fast

হিপ সার্চ O(n), জায়গা Minimal

সিকোয়েনশিয়াল: সার্চ Fast

সিকোয়েনশিয়াল ইনসার্ট Slow

হ্যাশড: `h(k) = k mod N`

ইনডেক্সড: বাড়তি জায়গা, খুব দ্রুত

প্রাইমারি স্টোর = RAM, ভোলাপূর্ণ

সেকেন্ডারি = ডিস্ক, স্থায়ী

টার্শিয়ারি = টেপ, আর্কাইভ

বিট → বাইট → ব্লক → ফাইল → ডিবি

`bfr = [আকার ÷ রেকর্ডআকার]`

ব্লক = [রেকর্ড ÷ `bfr`]

প্রাইমারি ইনডেক্স: সার্চ কি

সেকেন্ডারি ইনডেক্স: নন-কি কলাম

ক্লাস্টার্ড: ডেটা সাজানো

নন-ক্লাস্টার্ড: শুধু পয়েন্টার

মাত্র একটিই ক্লাস্টার্ড ইনডেক্স

ডেন্স = প্রতি রেকর্ডে এন্ট্রি

স্পার্স = প্রতি ব্লকে এন্ট্রি

মাল্টিলেভেল = ইনডেক্সের উপর ইনডেক্স

B+ ট্রি: সব কি লিভে

B+ লিভ কি-ক্রমে শৃঙ্খলবদ্ধ

B+ সার্চ / ইনসার্ট = O(log n)

B+ রেঞ্জ = O(log n) + k

হ্যাশ সমতা গড়ে O(1)

হ্যাশ: **রেঞ্জ কোয়েরি নেই**

SQL → পার্সার → অপটিমাইজার

→ এক্সিকিউশন প্ল্যান → ফলাফল

কস্ট = I/O + CPU + কমিউনিকেশন

হ্যাশ জয়েন: বড় · মার্জ: সাজানো

বাম-ডান প্রিফিক্স নিয়ম

কভারিং ইনডেক্স: টেবিল অপ্রশই নয়

MCQ-র জন্য সেরা ১২টি এক-লাইনার

1. হিপ = কোনো ক্রম নয় ⇒ ইনসার্ট দ্রুত, সার্চ O(n)
2. সিকোয়েনশিয়াল = সাজানো ⇒ সার্চ দ্রুত, লেখা ধীন
3. প্রাইমারি স্টোর (RAM) ভোলাপূর্ণ; ডিস্ক নয়
4. ব্লকিং ফ্যাক্টরে ফ্লোর, ব্লক সংখ্যায় সিলিং
5. একটি টেবিলে সর্বোচ্চ **একটি** ক্লাস্টার্ড ইনডেক্স
6. স্পার্স ইনডেক্স ⇒ প্রতি ব্লকে এন্ট্রি, সাজানো ফাইল
7. বি প্লাস ট্রিতে প্রতিটি কি **লিভে** বসে
8. লিভগুলো **শৃঙ্খলবদ্ধ** — তাই BETWEEN দ্রুত
9. বি প্লাস ট্রি **বাইনারি ট্রি নয়**
10. হ্যাশ =-এ O(1), BETWEEN-এ অচল
11. অপটিমাইজারের কস্ট = I/O + CPU + কমিউনিকেশন
12. কম্পোজিট ইনডেক্স a ও a,b দেয়, b একা নয়

সবচেয়ে বড় ফাঁদগুলো: রেঞ্জ দিয়ে **হ্যাশ ইনডেক্স** · একাধিক **ক্লাস্টার্ড ইনডেক্স** · বি প্লাস ট্রিতে কি **শুধু ইন্টারনাল নোডে** · বি প্লাস ট্রিকে বাইনারি ট্রি বলা · `h(key)=key mod N` **N ছাড়া** · **সিলিং** ভুলে ফ্লোর করা · অব্যবহৃত কলামে ইনডেক্স।

সব কৌশল একসঙ্গে: ডিস্কের ক্রম সার্চ কেনে আর insert বিক্রি করে · প্রাইমারি দ্রুত কিন্তু বিস্মৃতিশীল, ডিস্ক নিজের ঠিকানা, টেপ আটলার · ইনডেক্স হলো দামের ট্যাগ · ডেন্স লম্বা ও চওড়া, স্পার্স সরু ও লম্বা · শৃঙ্খলবদ্ধ লিভই রেঞ্জের উত্তর · =-এর জন্য হ্যাশ, BETWEEN-এর জন্য ট্রি।



তিন কাঠামো, একটি প্ল্যান

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. একটি **হিপ** (আনঅর্ডার্ড) ফাইল কেন বেছে নেওয়া হয়?
ক) যেকোনো অ্যাট্রিবিউটে সার্চ দ্রুত
খ) ইনসার্ট দ্রুত, কোনো ক্রম ধরে রাখার দরকার নেই
গ) এতে বাড়তি জায়গা লাগে না
ঘ) রেঞ্জ কোয়েরি সমর্থন করে

২. নিচের কোন স্টোরেজটি **ভোলাপূর্ণ**?
ক) সেকেন্ডারি (HDD/SSD)
খ) টার্শিয়ারি (টেপ)
গ) প্রাইমারি (র‍্যাম)
ঘ) কোনোটিই নয় — সবগুলোই স্থায়ী

৩. একটি টেবিলে সর্বোচ্চ কতটি **ক্লাস্টার্ড** ইনডেক্স থাকতে পারে?
ক) দুটি
খ) প্রতিটি কলামে একটি
গ) কেবল একটি
ঘ) যেকোনো সংখ্যক

৪. একটি **স্পার্স** প্রাইমারি ইনডেক্সে এন্ট্রি থাকে:
ক) প্রতি রেকর্ডে
খ) প্রতি ডেটা ব্লকে
গ) প্রতি অ্যাট্রিবিউটে
ঘ) প্রতি সেকেন্ডারি কি-তে

৫. বি প্লাস ট্রি `salary BETWEEN 20000 AND 30000`-এ কেন দক্ষ?
ক) সব কি লিভে থাকে এবং লিভগুলো কি-ক্রমে শৃঙ্খলবদ্ধ
খ) এর কোনো ইন্টারনাল নোড নেই
গ) এটি সব রেকর্ড রুটে রাখে
ঘ) এটি সবসময় বাইনারি ট্রি

৬. 100 বাইটের 30,000টি রেকর্ড, 1024 বাইটের ব্লকে unspanned, কি-র উপর সাজানো। **স্পার্স** ইনডেক্সে প্রতি ডেটা ব্লকে একটি 15 বাইটের এন্ট্রি। কত ব্লক লাগবে?
ক) 44
খ) 45
গ) 442
ঘ) 30

৭. `h(key) = key mod N`, যেখানে **N = 2**। কি 10, 20, 25 এর মান হবে:
ক) 1, 1, 0
খ) 0, 1, 0
গ) 2, 2, 1
ঘ) 0, 0, 1

৮. হ্যাশ ইনডেক্স কোন কোয়েরির উত্তর সবচেয়ে ভালো দেয়?
ক) `salary BETWEEN 20000 AND 30000`
খ) `ORDER BY salary`
গ) `WHERE empid = 4471`
ঘ) `SELECT MIN(salary)`

৯. `(a, b)`-এর উপর কম্পোজিট ইনডেক্স কী করতে পারে?
ক) শুধু `WHERE b = ?`
খ) `WHERE a = ?` এবং `WHERE a=? AND b=?`
গ) শুধু `ORDER BY b`
ঘ) কিছুই নয় — কম্পোজিট অব্যবহার্য

১০. কস্ট-বেজড অপটিমাইজার কী অনুমান করে?
ক) I/O, CPU ও কমিউনিকেশন খরচ
খ) শুধু ফেরত দেওয়া সারির সংখ্যা
গ) টেবিলের আকার ও কলাম সংখ্যা
ঘ) টেবিলগুলোর বর্ণানুক্রম

উত্তরমালা → ১-খ · ২-গ · ৩-গ · ৪-খ · ৫-ক · ৬-খ · ৭-ঘ · ৮-গ · ৯-খ · ১০-ক

লিখিত প্রশ্ন: ফাইল অর্গানাইজেশনের সংজ্ঞা দাও এবং চার প্রকারের Insert / Delete / Search / Space-এর প্রোফাইল দাও। একটি ডেটা ফাইলে **100 বাইটের 30,000টি রেকর্ড**, **1024 বাইটের ব্লকে** unspanned, কি-র উপর সাজানো। **স্পার্স** প্রাইমারি ইনডেক্সে প্রতি ডেটা ব্লকে একটি **15 বাইটের** এন্ট্রি। কত ব্লক লাগবে? প্রতিটি ধাপ দেখাও, ডেন্স ইনডেক্সের সঙ্গে তুলনা করো, এবং BETWEEN-এ কেন বি প্লাস ট্রি হ্যাশের চেয়ে ভালো তা বলো। ১০ নম্বর

১ **চারটি অর্গানাইজেশন (২)** — **হিপ**: কোনো ক্রম নেই, insert Fast, delete Moderate, search Slow O(n), জায়গা Minimal। **সিকোয়েনশিয়াল**: সাজানো, insert Slow (রেকর্ড সরে), delete Slow, search Fast, Minimal। **হ্যাশড**: শুধু search Fast, বাকি Moderate। **ইনডেক্সড**: search Very Fast, জায়গা Extra। ২

২ **ডেটা ব্লকিং ফ্যাক্টর (২)** — unspanned মানে কোনো রেকর্ড ব্লক পেরিয়ে যায় না: `bfr = ⌊1024 ÷ 100⌋ = 10`, তাই ফাইলে `30000 ÷ 10 = 3000` ব্লক। ২

৩ **ইনডেক্স ব্লক (২)** — স্পার্স ⇒ 3000 এন্ট্রি; `bfr = ⌊1024 ÷ 15⌋ = 68` প্রতি ব্লকে; ব্লক = `⌈3000 ÷ 68⌉ = ⌈44.1⌉ = 45` — সিলিং লাগে কারণ শেষ ব্লক আংশিক ভর্তি। **ডেন্স** ইনডেক্স ⇒ `⌈30000 ÷ 68⌉ = 442` ব্লক। ২

৪ **রেঞ্জে বি প্লাস ট্রি বনাম হ্যাশ (৩)** — প্রতিটি কি তার রেকর্ড পয়েন্টারসহ **লিভে** বসে এবং লিভগুলো **কি-ক্রমে শৃঙ্খলবদ্ধ**: 20000-এর প্রথম লিভে **O(log n)** খরচে নামো, তারপর চেইন ধরে এগিয়ে যাও যতক্ষণ না 30000 পেরোয় — খরচ O(log n) + k। ইন্টারনাল নোডে শুধু কি ও পয়েন্টার; ফ্যান-আউট বড় বলে এটি বাইনারি ট্রি নয়। হ্যাশ ইনডেক্স সরাসরি `h(salary)`-এ লাফায়: =-এ O(1), কিন্তু কি কোনো **ক্রমে** থাকে না, তাই BETWEEN-এ প্রতিটি বাকেট পড়তে হয় — **রেঞ্জে ট্রি, সমতায় হ্যাশ**। ৩

৫ **নিয়মগুলো যোগ করো (১)** — একটি টেবিলে সর্বোচ্চ **একটি** ক্লাস্টার্ড ইনডেক্স; স্পার্স ইনডেক্সের জন্য **সাজানো** ফাইল লাগে; কম্পোজিট ইনডেক্স **বাম-ডান প্রিফিক্স** নিয়ম মানে। ১

পরীক্ষার আগে ✓

- ✓ চারটি ফাইল অর্গানাইজেশনের নাম বলতে পারি
- ✓ চার-কলামের টেবিলটি হুবহু বলতে পারি
- ✓ ব্লকিং ফ্যাক্টর হিসাব করতে পারি
- ✓ লিভ চেইনসহ বি প্লাস ট্রি আঁকতে পারি
- ✓ ডেন্স আর স্পার্স আলাদা করতে পারি
- ✓ কোয়েরির পাঁচটি ধাপ বলতে পারি